

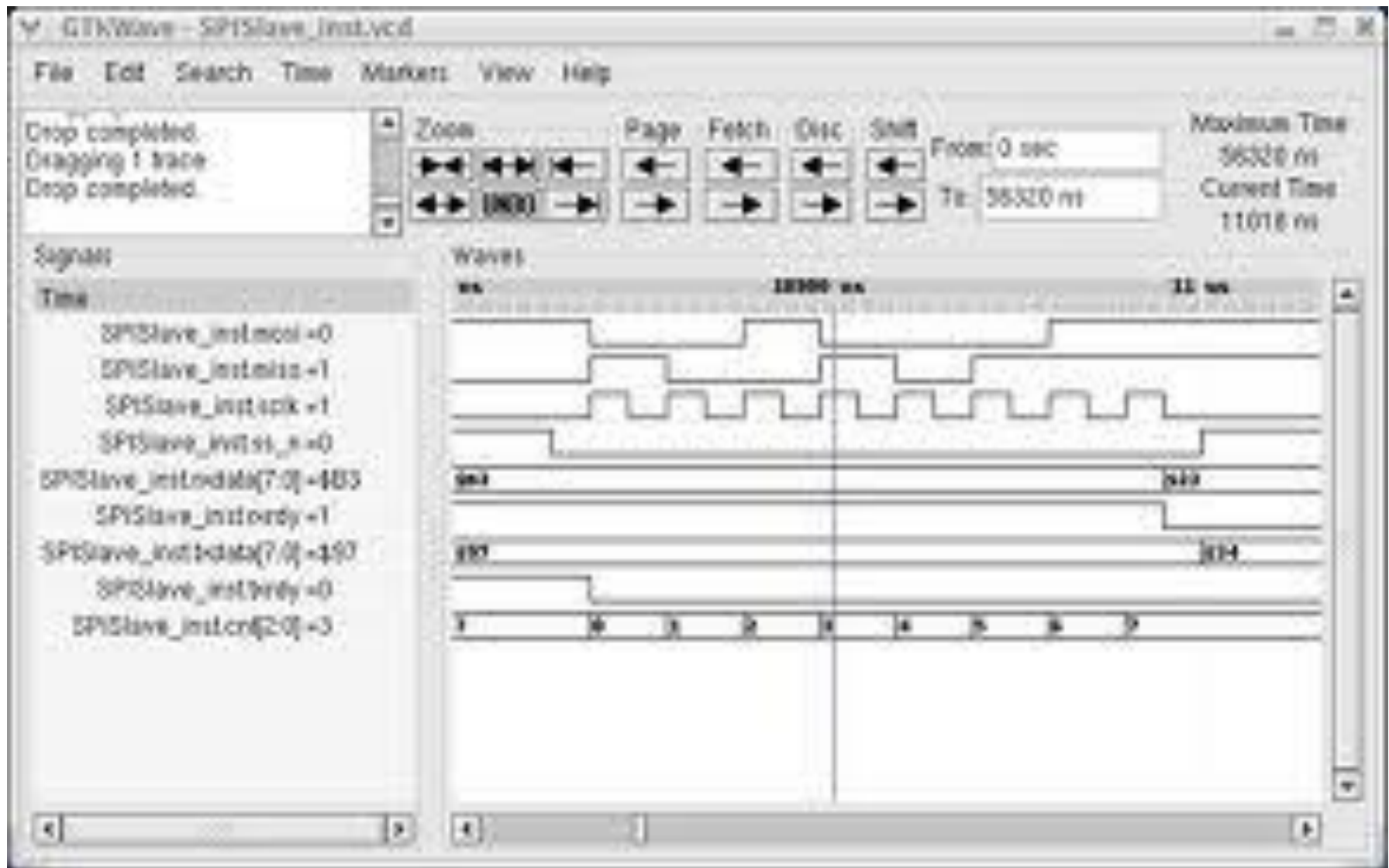
בדיקת מערכות ספרתיות ואימות תכנון

פרק 3 : סימולציה

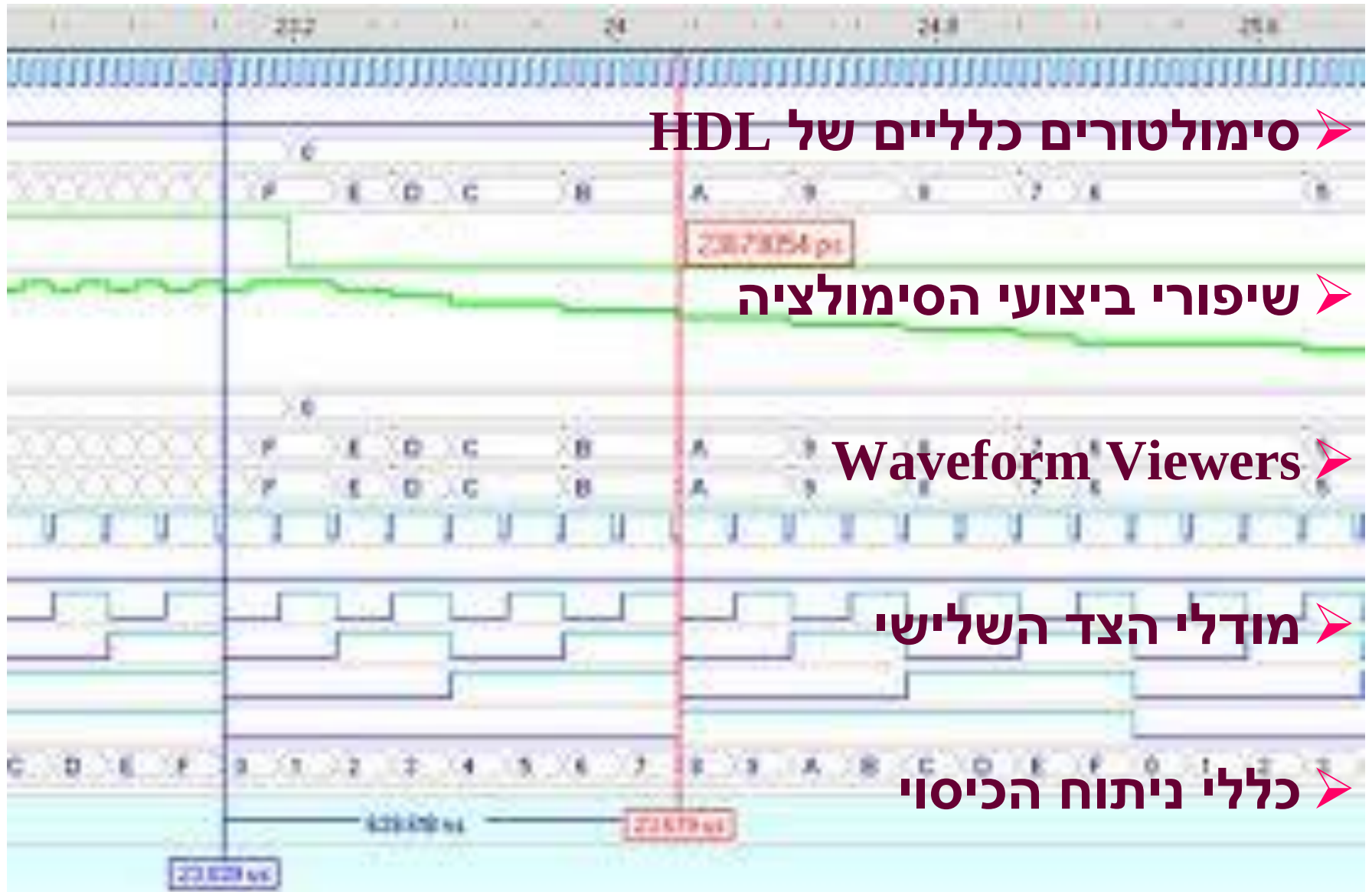
חלק 2



HDL



כלים לאימות פונקציונאלית



סימולטורים כלליים של HDL

➤ סימולטורים הפופולרים ביותר בתעשייה

➤ "EVENT DRIVEN" אלגוריתמים

➤ **Mentor Graphics ModelSim - MTI VSIM**

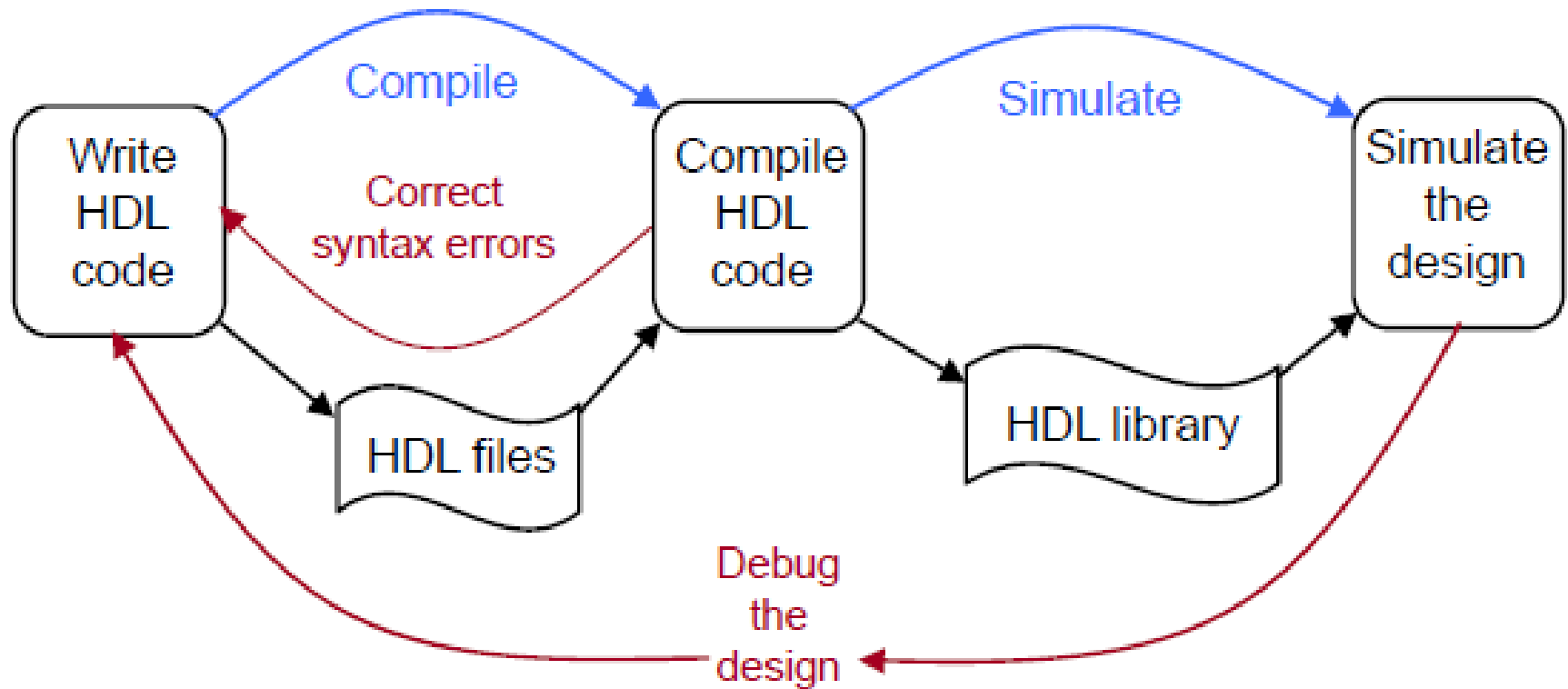
➤ **Cadence NCSim - Verilog XL**

➤ **Synopsys VCS**

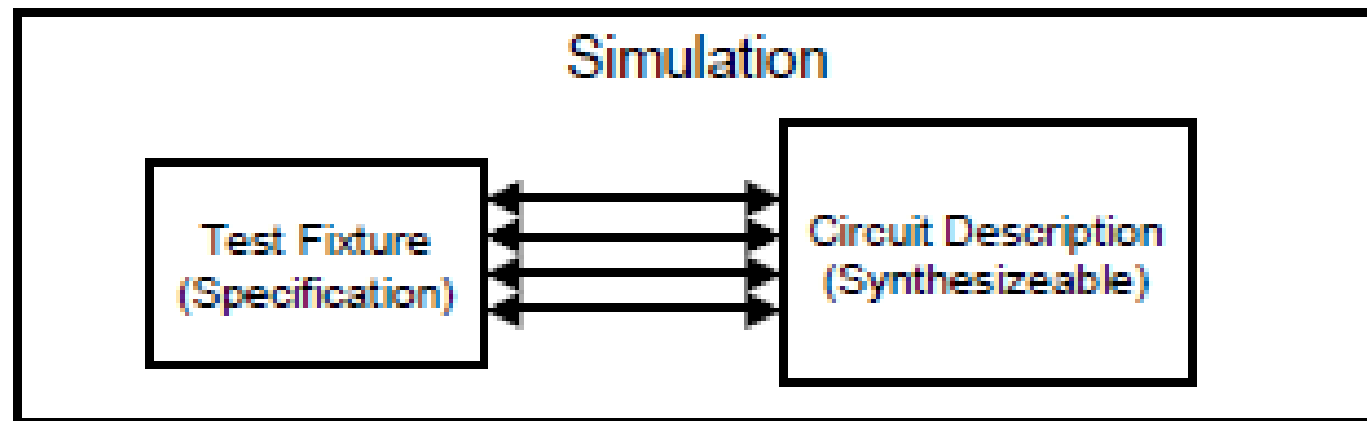
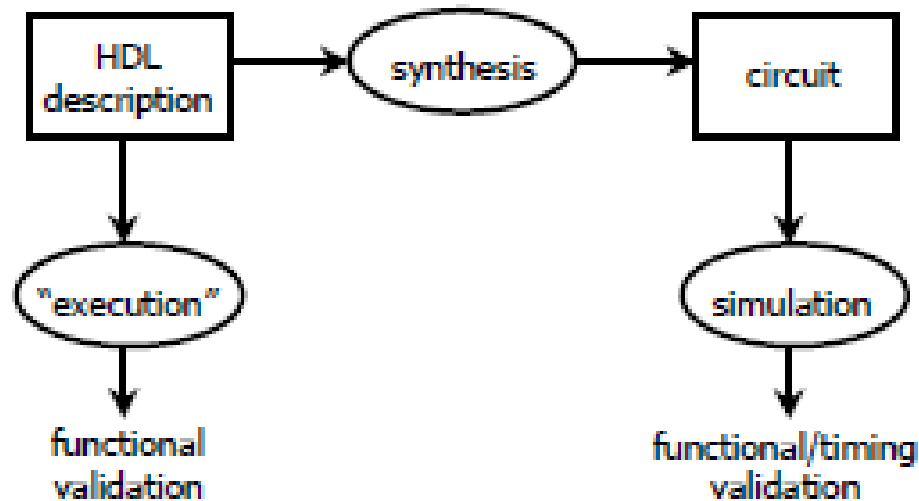
➤ VHDL's execution model is defined in detail in the IEEE LRM (Language Reference Manual)

➤ Verilog's execution model is defined by Cadence's Verilog-XL simulator ("reference implementation")

ModelSim



סימולציה וסינטזה



מידול טרנזיסטור



מודלים בסיסיים לסימולציה -- BSIM

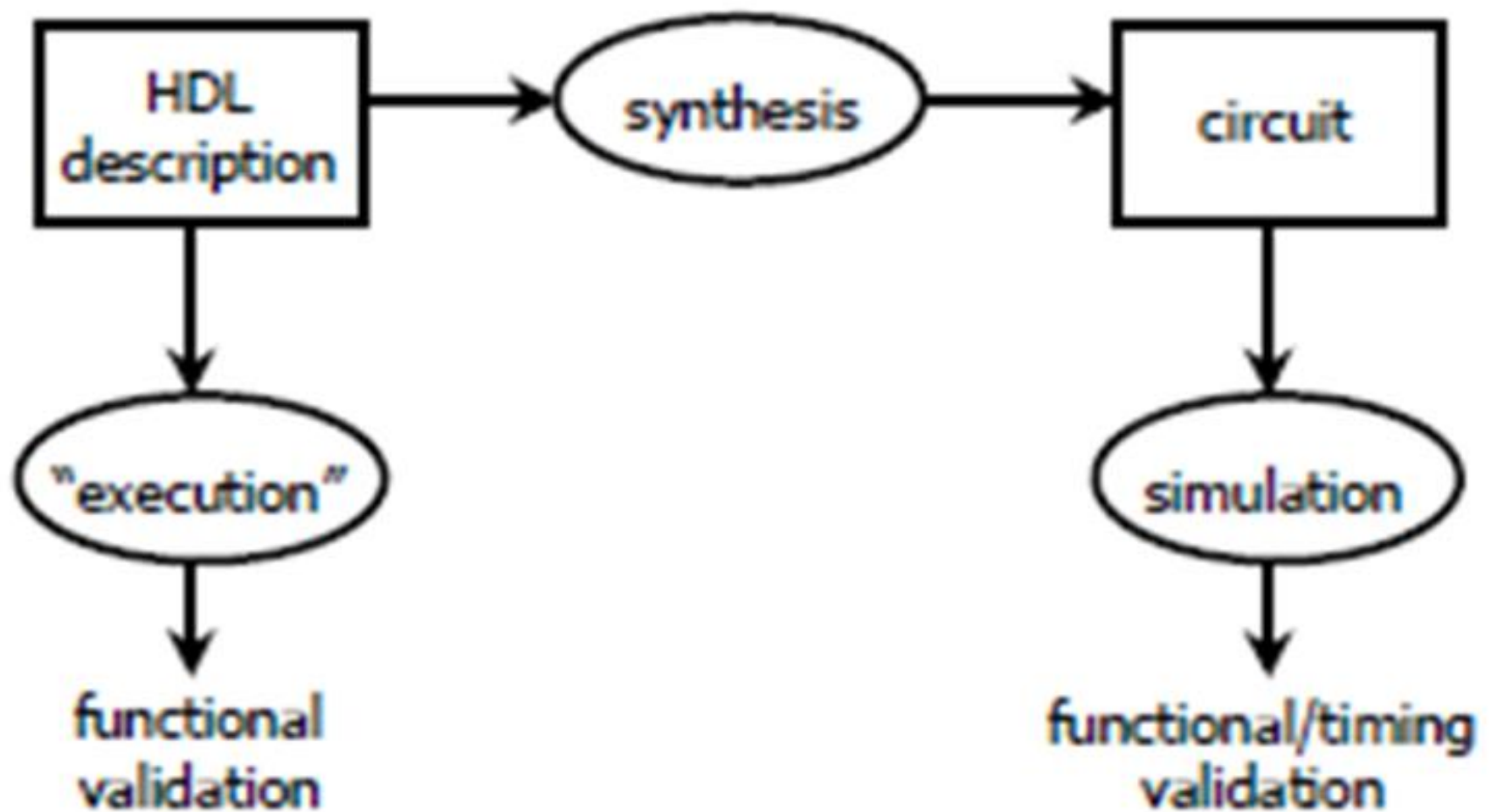
Company	Internal Use	Customer Use
Intel	Modified	Yes
IBM	BSIM3	Yes
Nat. Semi	Yes	Yes?
AMD	Yes?	Yes
Analog Dev.	Yes	Yes
TI	Yes	Yes
NEC	Yes	Yes
Siemens	Yes	Yes
DEC	Yes	Yes
HP	Yes	Yes?
Lucent	No	Yes?
Rockwell	No	Yes
TSMC	Yes?	Yes?
Motorola	Yes?	Yes
Samsung	Yes	Yes?
Hyundai	No	Yes
Chartered	Yes	Yes
	Yes	

מי משתמש ב-BSIM3v3? (Oct. 10, 1997)

These are the only versions of BSIM4 that have been released by BSIM Research Group at UC Berkeley:

- A. BSIM4.0.0 released on 03/24/2000
- B. BSIM4.1.0 released on 10/11/2000
- C. BSIM4.2.0 released on 04/06/2001
- D. BSIM4.2.1 released on 10/05/2001
- E. BSIM4.3.0 released on 05/09/2003
- F. BSIM4.4.0 released on 03/04/2004
- G. BSIM4.5.0 released on 07/29/2005
- H. BSIM4.6.0 released on 12/13/2006
- I. BSIM4.6.1 released on 05/18/2007
- J. BSIM4.6.2 released on 07/31/2008
- K. BSIM4.6.3 released on 09/19/2008
- L. BSIM4.6.4 released on 04/17/2009
- M. BSIM4.6.5 released on 09/09/2009
- N. BSIM4.7.0 released on 04/08/2011

סימולציה וסינטזה



מודלים בסיסיים לסימולציה -- BSIM

BSIM4.6.4 Developers:

- Professor Chenming Hu (**project director**), UC Berkeley
- Professor Ali M. Niknejad(**project director**), UC Berkeley
- Wenwei Yang, UC Berkeley
- Darsen Lu, UC Berkeley

מודלים בסיסיים לסימולציה -- BSIM

Developers of BSIM4 Previous Versions:

- Dr. Weidong Liu, Synopsys
- Dr. Xiaodong Jin, Marvell
- Dr. Kanyu (Mark) Cao, UC Berkeley
- Dr. Jeff J. Ou, Intel
- Dr. Jin He, UC Berkeley
- Dr. Xuemei (Jane) Xi, UC Berkeley
- Mohan V. Dunga, UC Berkeley
- Professor Ali M. Niknejad, UC Berkeley
- Professor Chenming Hu, UC Berkeley

מודלים בסיסיים לסימולציה -- BSIM

➤ Web Sites:

BSIM4 web site with BSIM source code and documents:

<http://wwwdevice.eecs.berkeley.edu/~bsim3/bsim4.html>

➤ Compact Model Council:

<http://www.eigroup.org/CMC/default.htm>

SPICE Model Parameters

הפרמטרים של מודל BSIM4 ניתן לחלק לכמה קבוצות.

➤ הפרמטרים העיקריים במודל משמשים למידול השפעות פיזיות בסיסיות בטמפרטורת חדר. הם מחולקים לקבוצות קטנות הקשורות לתופעות פיזיות של טרנזיסטור MOS

➤ הקבוצה השניה של הפרמטרים הם פרמטרים הקשורים בתהליך הייצור.

SPICE Model Parameters

➤ הקבוצה השלישית של פרמטרים קשורה לפרמטרים של טמפרטורה.

➤ שתי הקבוצות נוספות משמשות למידול התנהגות I_{AC} רעש של טרנזיסטור MOS

➤ הקבוצה האחרונה כוללת דגלים כדי לבחור מצבים מסוימים של פעולות, ומשתמשת להגדרה הפרמטרים במודל

סימולציה זו אימות מבוססת אירועים



➤ סימולציה של התכנון (לא יישום) לפני

הייצור

➤ סימולציה של התכנון מסתמכת על

הפשטות

➤ אתגר האימות: "איזה צירופים

שלקלט לספק לתכנון תחת האימות?"

➤ סימולציה דורש **stimulus**. היא דינמית, לא פשוט סטטית!

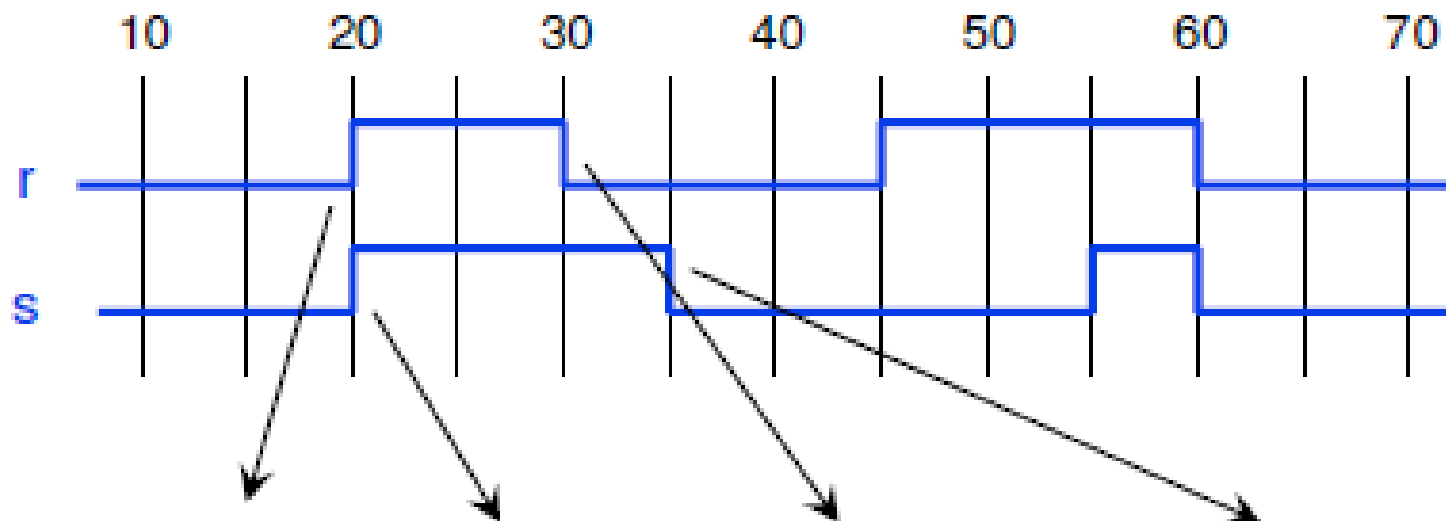
➤ סימולציה דורשת לשחזר את הסביבה שבה התכנון יהיה

בשימוש.

➤ פלטי הסימולציה מאומתים חיצונית נגד כוונת התכנון

מה זה בדיוק אירוע?

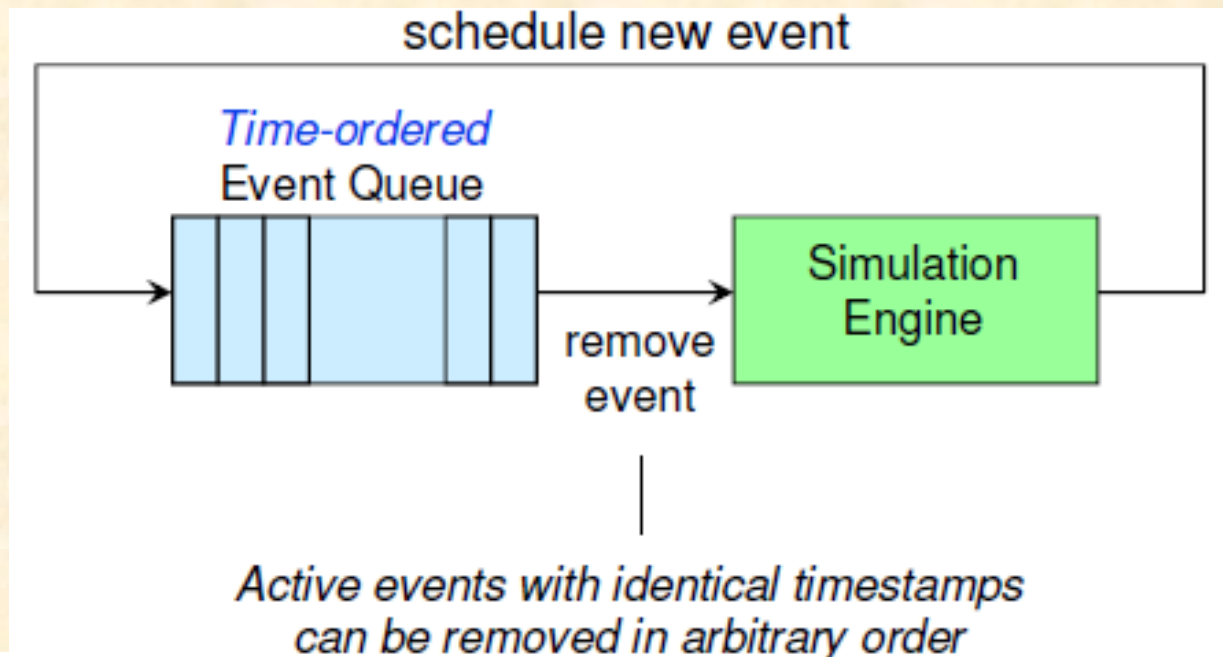
➤ Event = tuple (activity, simulation time)



Events: (r_rises, 20), (s_rises, 20), (r_falls, 30), (s_falls, 35)

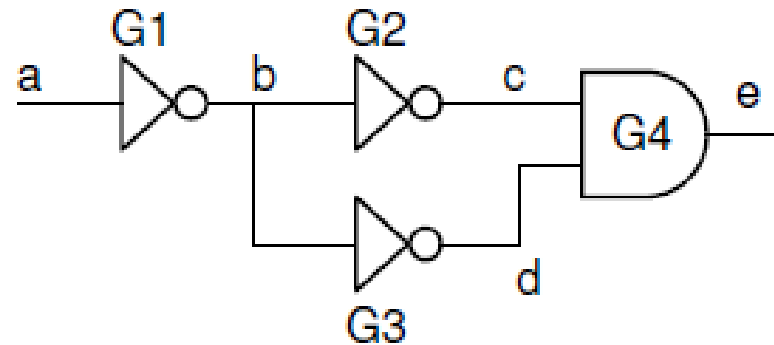
Verilog הוא סימולטור מבוסס אירועים

- סימולטור Verilog שומר רשימה ממוספרת של כל האירועים בעתיד: תור האירועים
- מנוע סימולציה קורא את האירועים מהתור, מבצעת אותם, ומוסיף אירועים חדשים על פי הצורך



דוגמה לריצת Verilog

```
initial begin
  #20 a = 1;
end
```



All gates have
propagation delay
of 5 units

Event Queue

empty

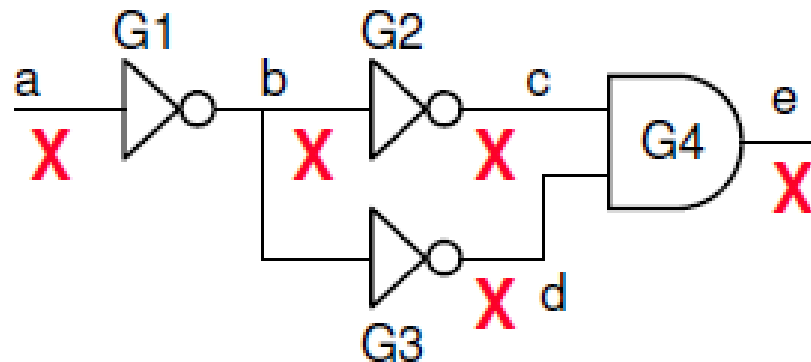
T = 0

Initialize simulation

All nets are at X

דוגמה לריצת Verilog

T=0



All gates have
propagation delay
of 5 units

Event Queue

```
update  
a = 1  
@T = 20
```

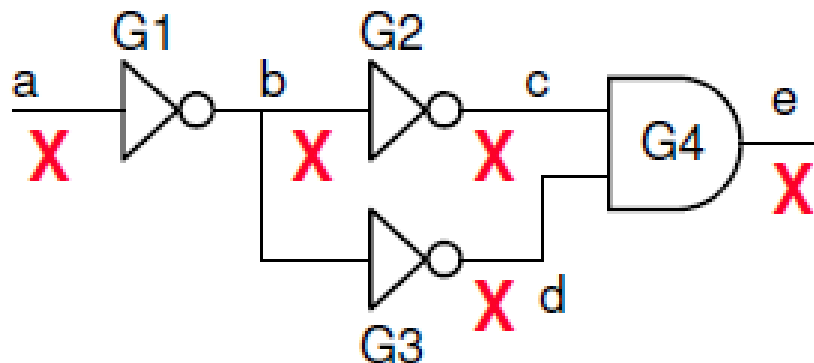
T = 0

Initialize simulation

All nets are at X

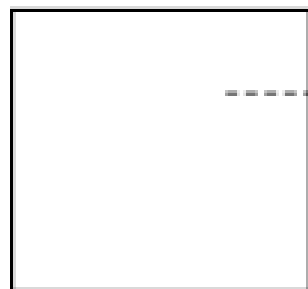
דוגמה לריצת Verilog

T=0



All gates have
propagation delay
of 5 units

Event Queue



update
a = 1
@T = 20

Remove event from Q. Event T = 20

Adjust simulation time to T = 20

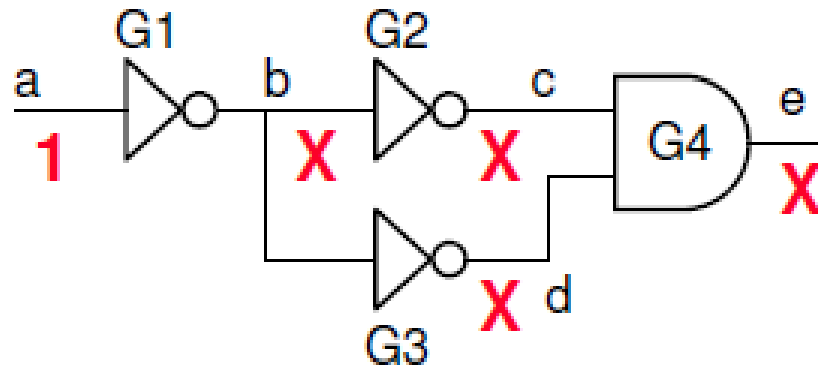
Update value of a

Fanout of a = {G1}

Execute G1. Output b should change to 0 at T = 20 + 5 = 25

דוגמה לריצת Verilog

T=20



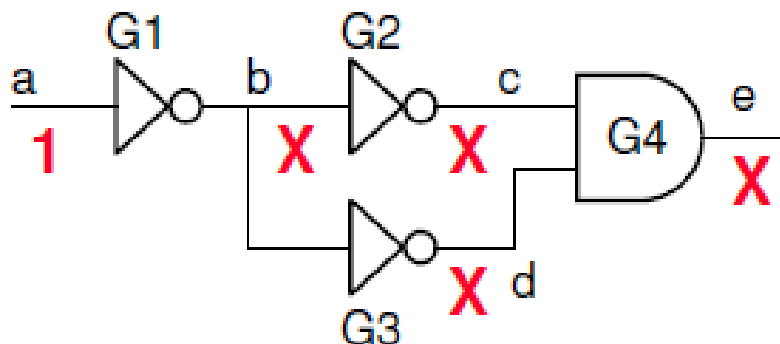
All gates have
propagation delay
of 5 units

Event Queue

update
b = 0
@T = 25

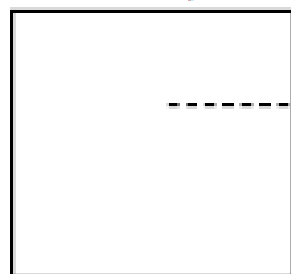
דוגמה לריצת Verilog

T=20



All gates have
propagation delay
of 5 units

Event Queue



update
b = 0
@T = 25

Remove event from Q. Event T = 25

Adjust simulation time to T = 25

Update value of b

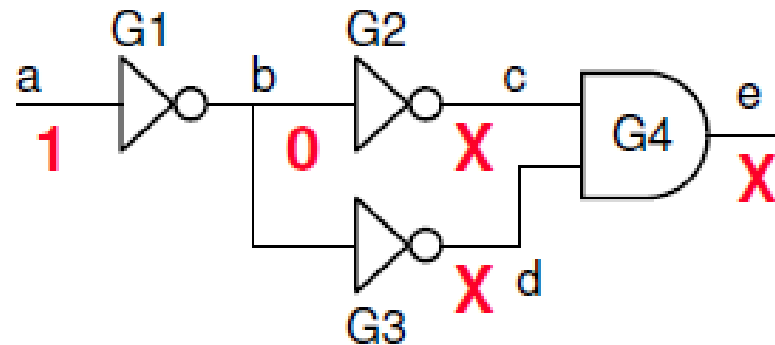
Fanout of b = {G2, G3}

Execute G2. c should change to 1 at T = 25 + 5 = 30

Execute G3. d should change to 1 at T = 25 + 5 = 30

דוגמה לריצת Verilog

T=25



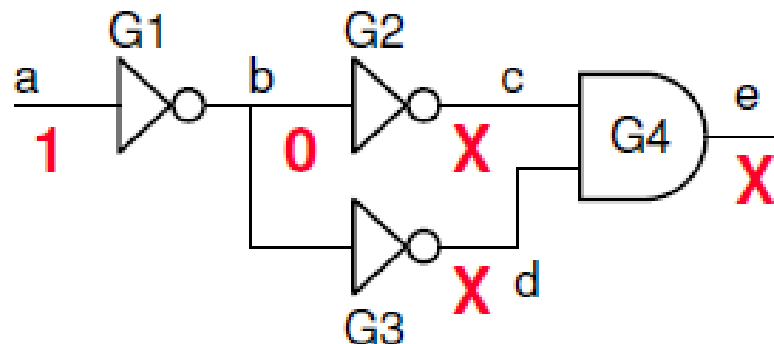
All gates have
propagation delay
of 5 units

Event Queue

update c = 1 @T = 30	update d = 1 @T = 30
----------------------------	----------------------------

דוגמה לריצת Verilog

T=25



All gates have
propagation delay
of 5 units

Event Queue

update c = 1 @T = 30	
----------------------------	--

update
d = 1
@T = 30

*Note: It is equally
valid to choose update-c
event. Event processing
order for equal timestamps
is not guaranteed.*

Remove event from Q. Event T = 30.

Adjust simulation time to T = 30

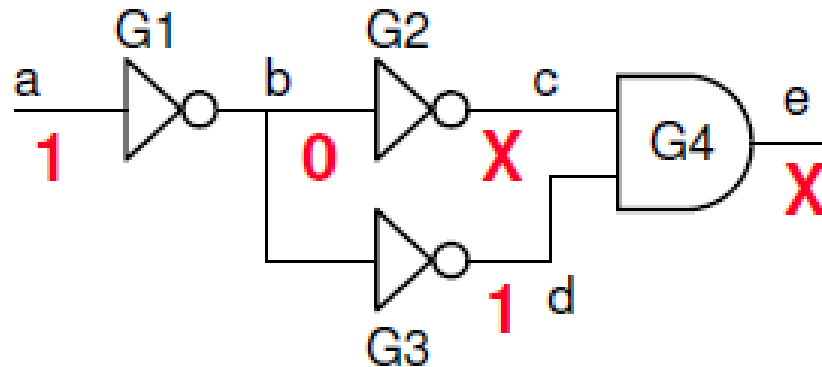
Update value of d

Fanout of d = {G4}

Execute G4. e will remain at X. No new event.

דוגמה לריצת Verilog

T=30



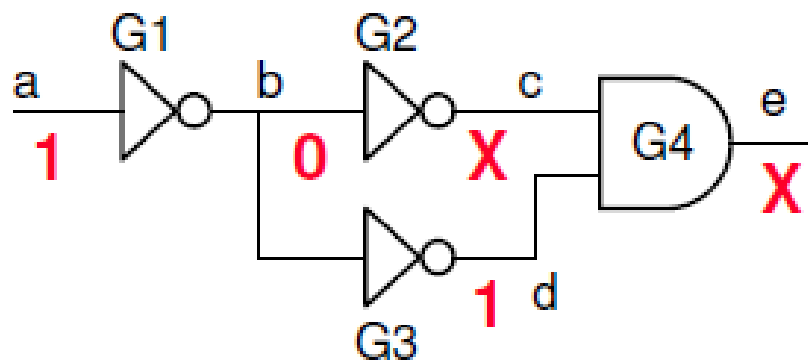
All gates have
propagation delay
of 5 units

Event Queue

```
update  
c = 1  
@T = 30
```

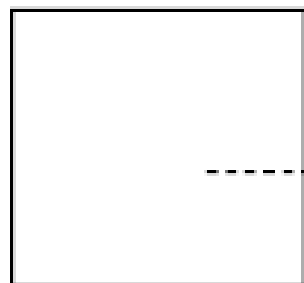
דוגמה לריצת Verilog

T=30



All gates have
propagation delay
of 5 units

Event Queue



update
c = 1
@T = 30

Remove event from Q. Event T = 30.

Simulation time remains at 30.

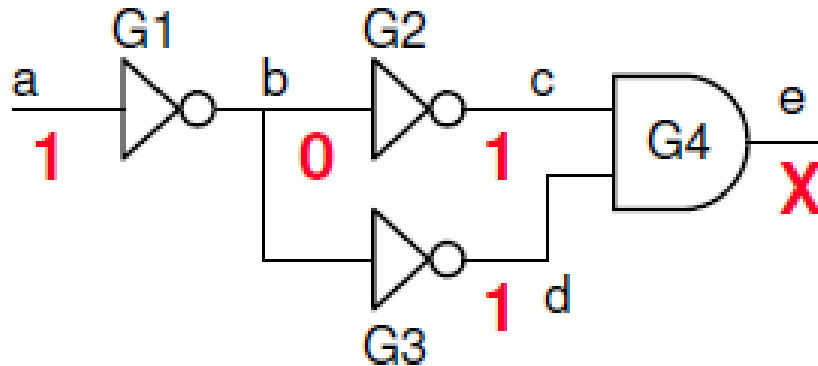
Update value of c

Fanout of c = {G4}

Execute G4. e should change to 1 at $T = 30 + 5 = 35$.

דוגמה לריצת Verilog

T=30



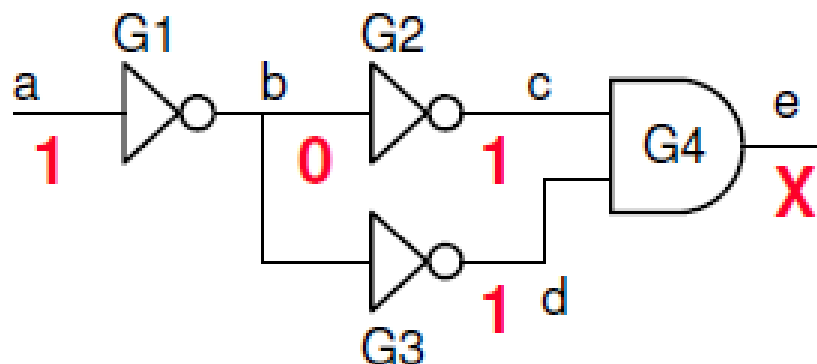
All gates have
propagation delay
of 5 units

Event Queue

update
e = 1
@T = 35

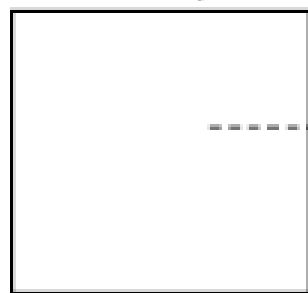
דוגמה לריצת Verilog

T=30



All gates have
propagation delay
of 5 units

Event Queue



update
e = 1
@T = 35

Remove event from Q. Event T = 35.

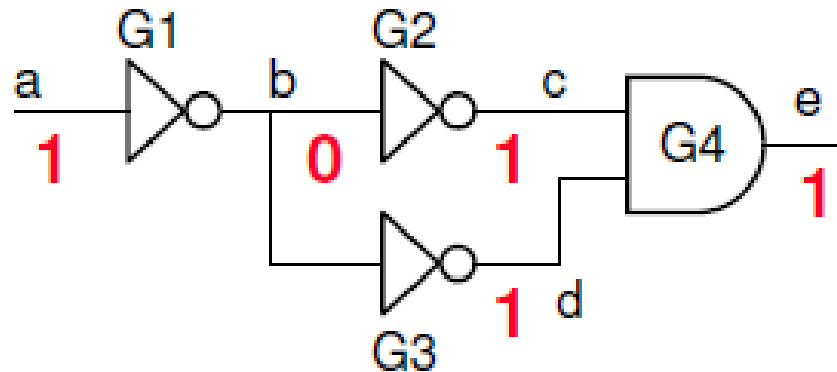
Simulation time adjusts to 35.

Update value of e

Fanout of e = { }

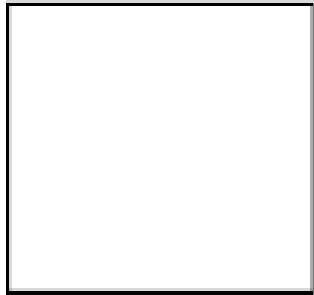
דוגמה לריצת Verilog

T=35



All gates have
propagation delay
of 5 units

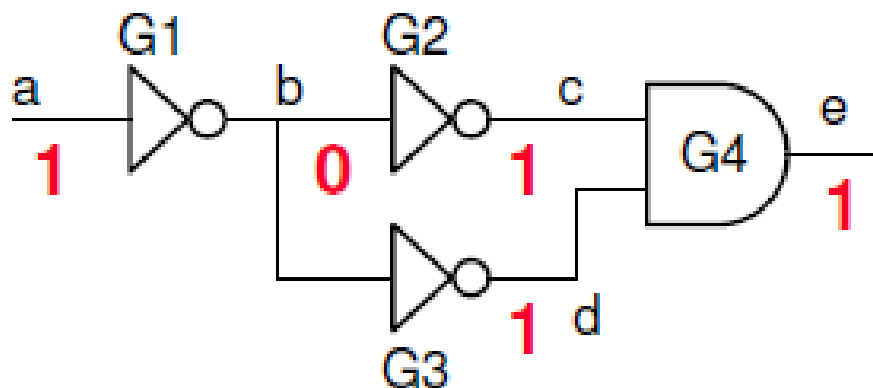
Event Queue



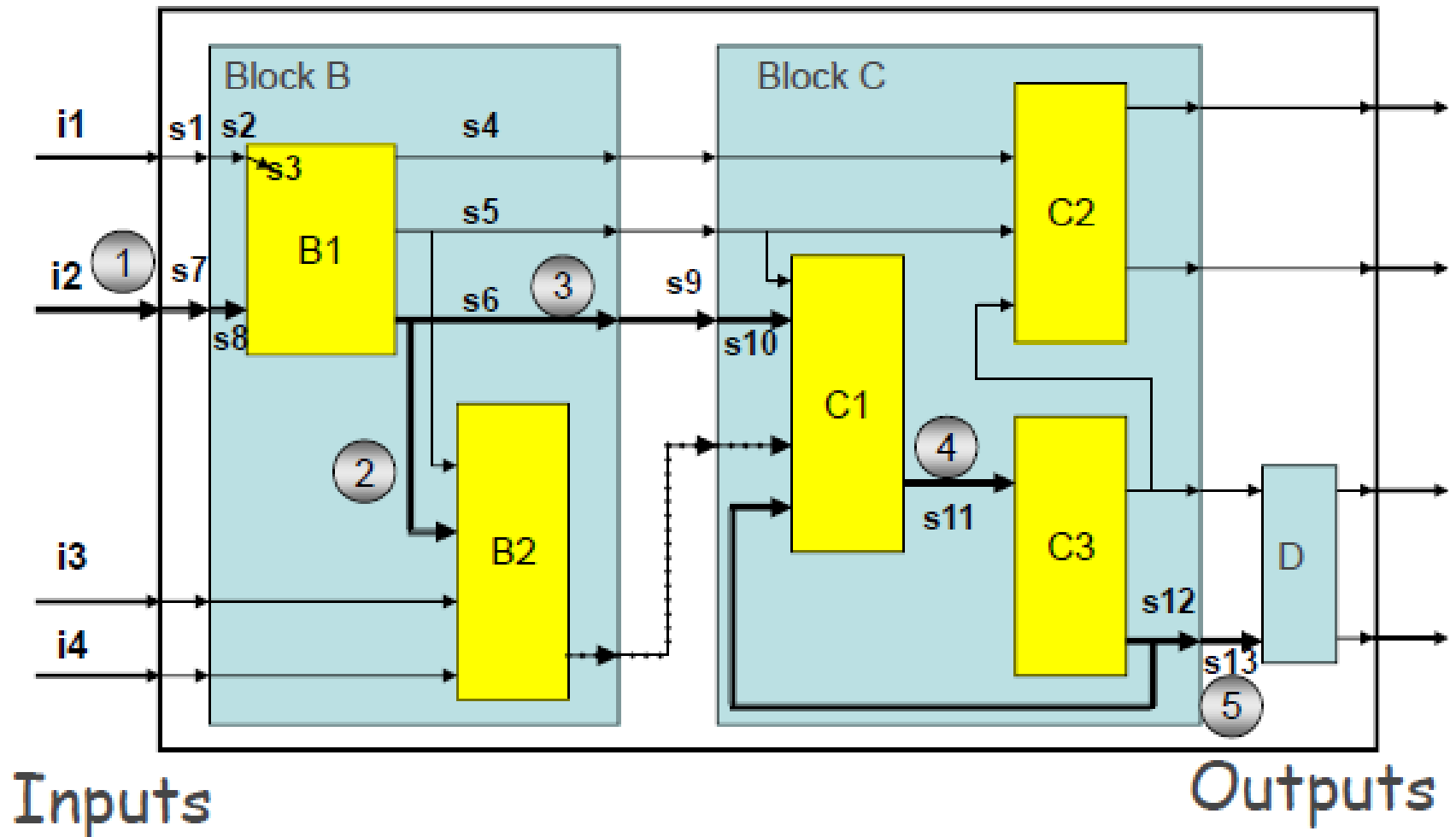
No new events. Done.

סימולציה ברמת השער

➤ אם חוט (wire) מתעדכן, אז כל השערים המחוברים לחוט מתבצעים המשמעות האמיתית של ביצוע השער היא: ליצור עדכון האירוע על חוט הפלט של השער



סימולציה מבוססת אירועים (דוגמא)



סימולציה מבוססת אירועים

האירוע הוא שינוי בקלט

➤ פלט הוא פונקציה של קלט

- הפלט משתנה רק כאשר קולטים משתנים

- אירוע גורם לסימולטור לחשב מחדש את ערכי הפלט

- פלטי של בלוק אחד ניתן לשימוש כקלט של בלוק אחר

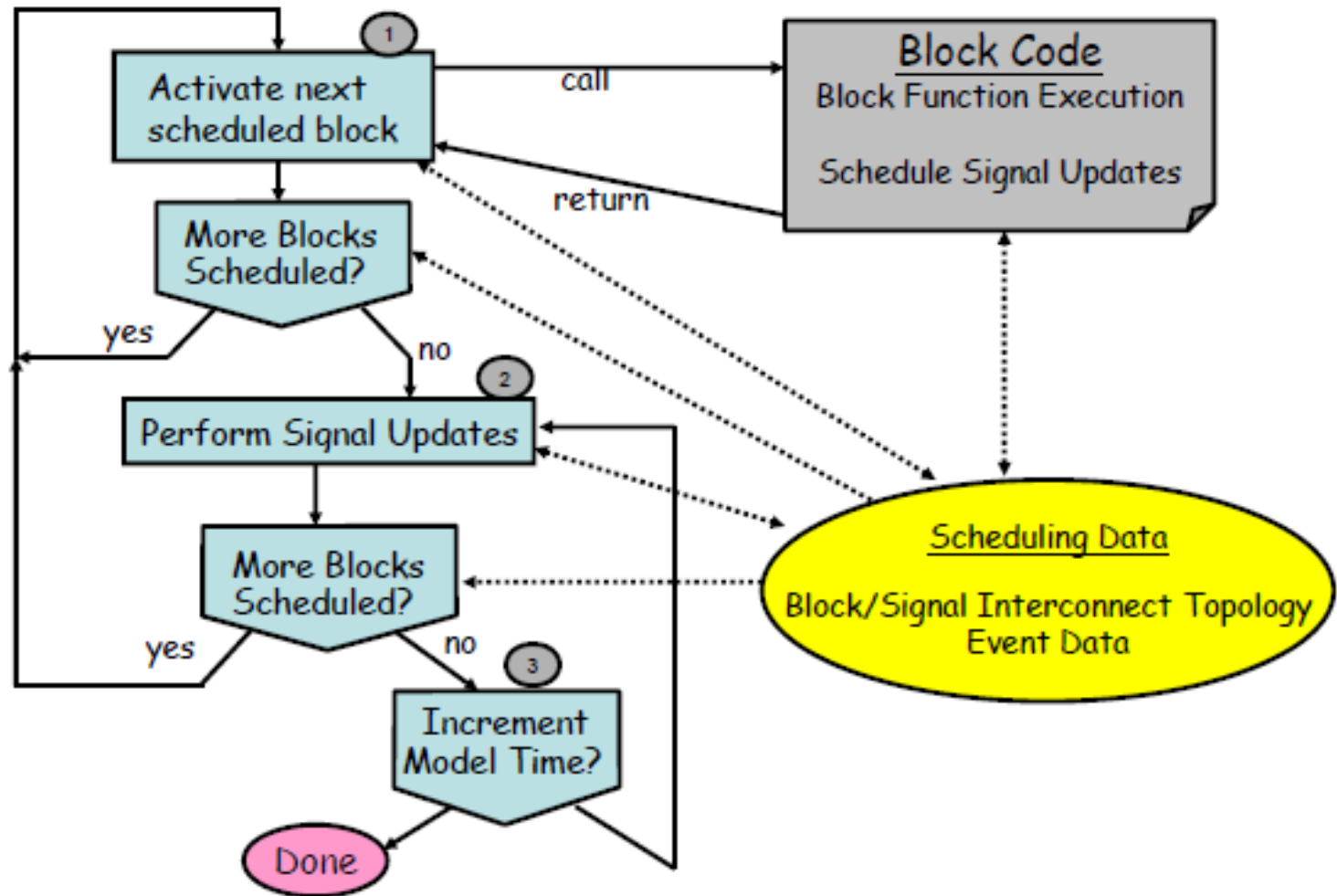
➤ החישוב מחדש ממשיך עד שאין שינויים בתכנון

עקרונות סימולציות מבוססות אירועים

סימולטור אירועים שומר רשימות רבות

- רשימה של כל בלוקי הפעלה אטומיים
 - מבנה הנתונים המייצג את הקישוריות של בלוקים בעזרת אותות
 - תור הזמן -נקודות זמן, כאשר אירועים קורים
 - תור האירועים
- הסימולטור חייב לעבד את כל התורים בזמן הסימולציה.

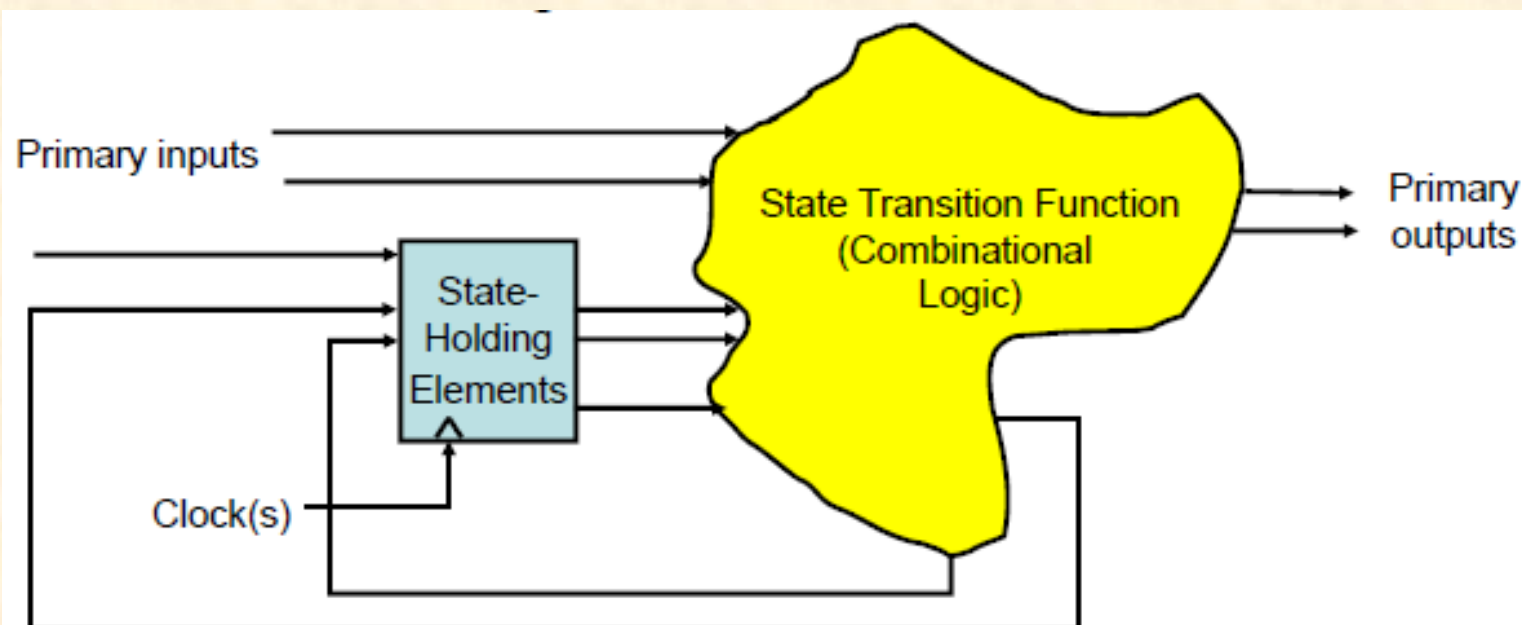
מנוע סימולציות מבוססות אירועים



סימולציה מבוססת מחזור שעות

מתודולוגיה של תכנון סינכרוני מבוססת אל תיור של תכנון שכולל:

1. אלמנטים ששומרים מצב -- State-holding elements
2. לוגיקה עבור פונקצית המעברים בין המצבים



בניית מודלים מבוססים מחזור שעון

➤ קומפיילר לשפה - כמו תהליך הסינתזה

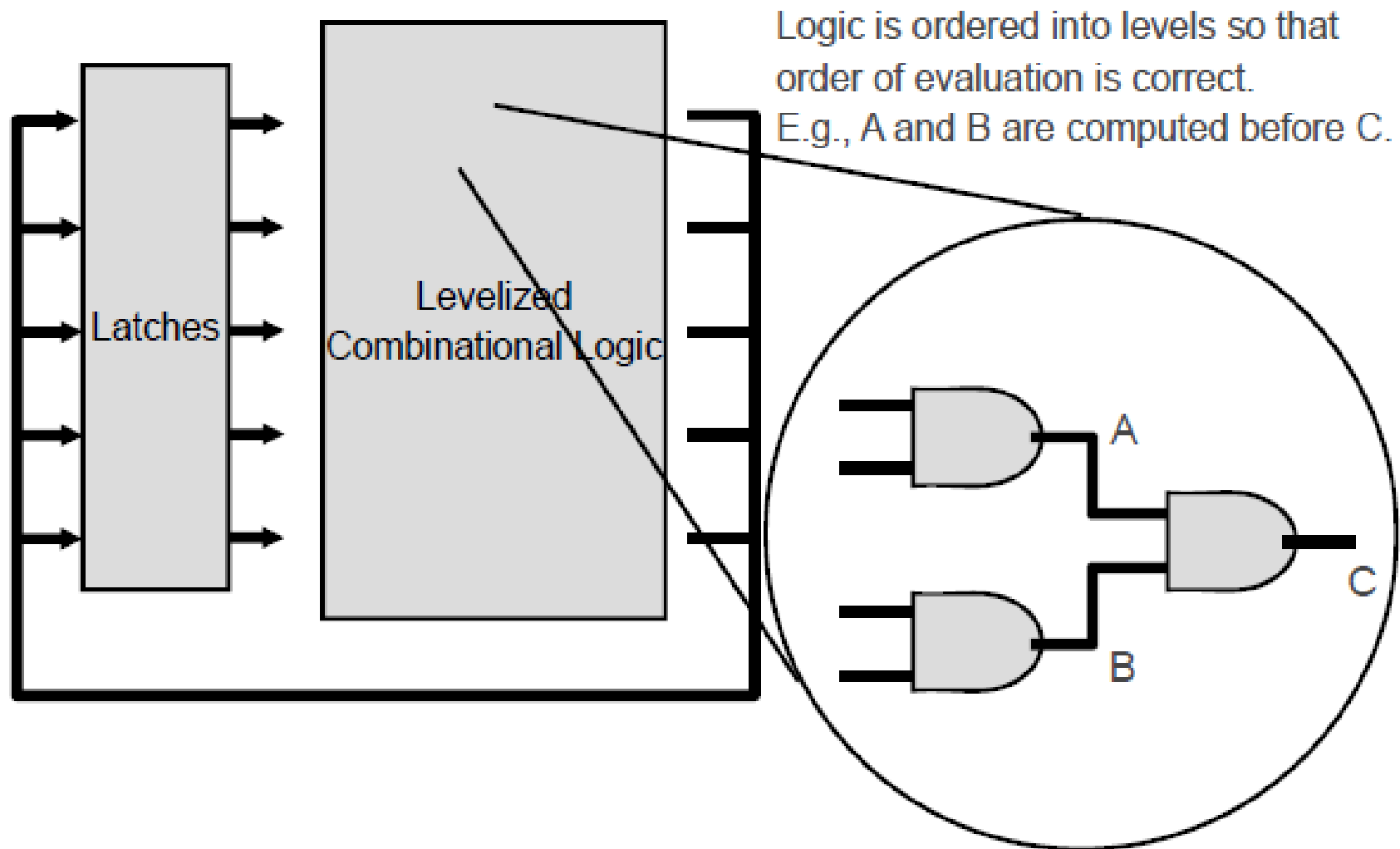
• לוגיקה ממופה לרשת אי - מחזורית של פונקציות
בוליאניות

➤ אופטימיזציה

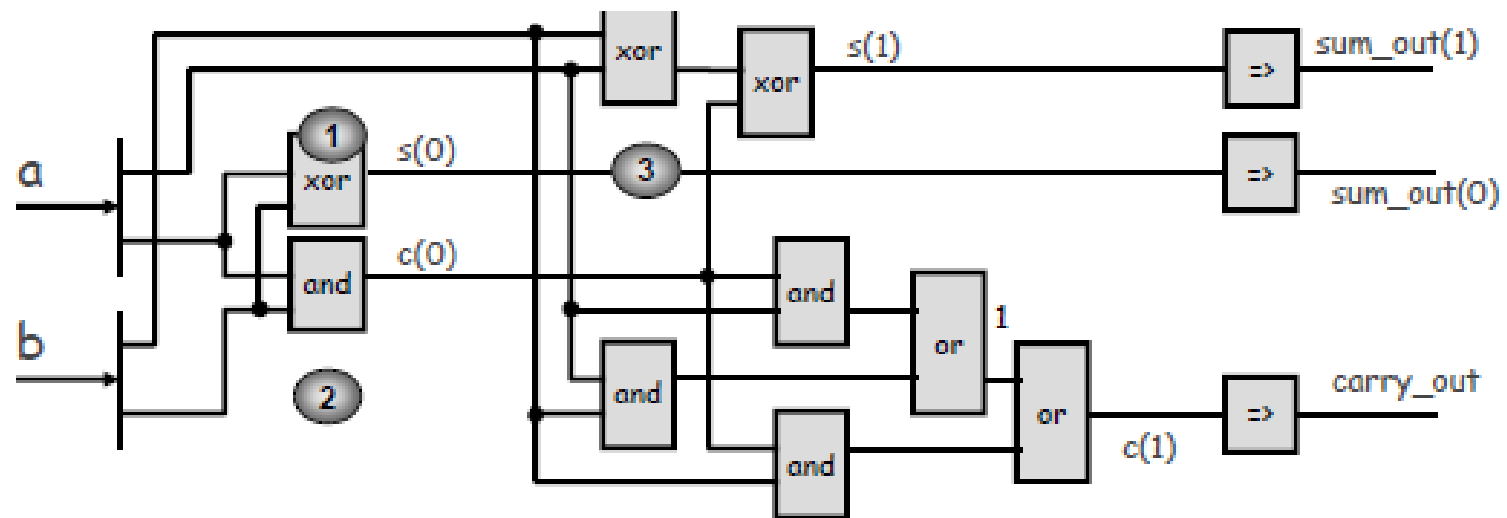
➤ תרגום לפקודות



מודלים מבוססים מחזור שעון



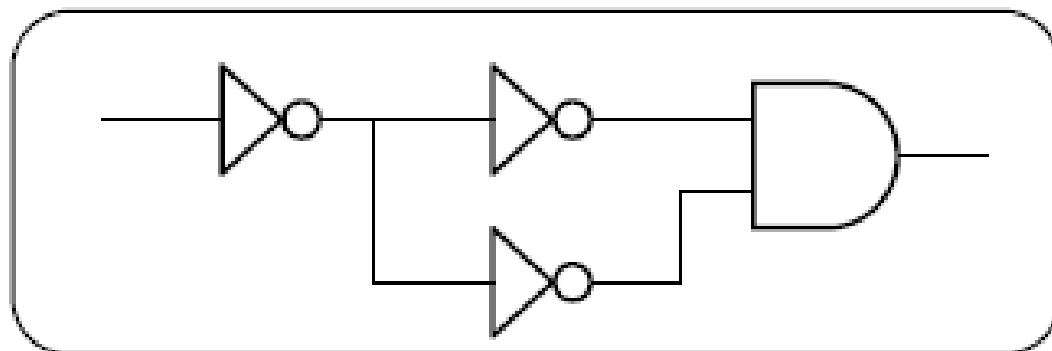
תרגום לפקודות



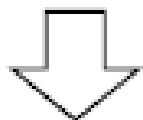
1 Load temp1, a(0)
 Load temp2, b(0)
 Xor temp1, temp2, temp3
 Store temp3, s(0)
 2 And temp1, temp2, temp3
 Store temp3, c(0)
 Load temp1, a(1)
 Load temp2, b(1)
 3 Xor temp1, temp2, temp3
 ...

We can cover every Boolean function
 into a minimal set (~4 or better)
 instructions

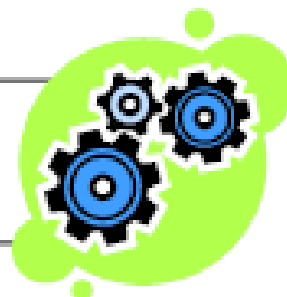
מנגנון סימולציה של Verilog



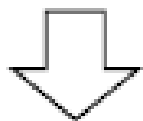
Verilog Model
of a hardware circuit
(many gates)



Modelsim
Verilog Simulator



Simulator creates the
illusion of parallel
activities in model



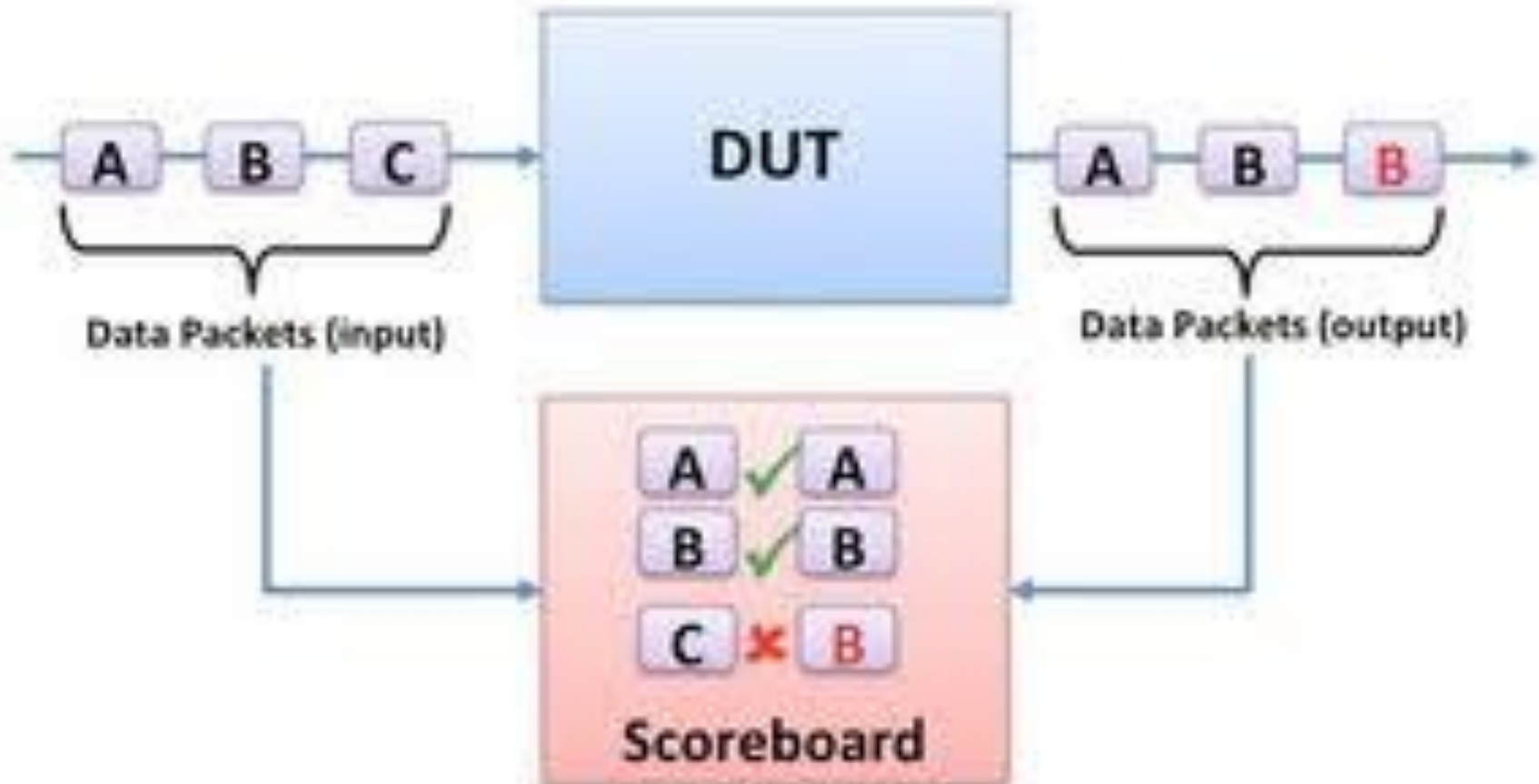
Sequential
Machine

Simulator is a software
program running on
a PC

סימולציה של HDL שונה מריצות C

- צריכים לבטא מקביליות (דברים קורים בו זמנית)
- צריכים למדל זמן: זמן סימולציה + מודל של מקביליות (אירועים, מחזורים, ...)
- צריכים למדל אורכי מילה שאינם סטנדרטיים
- צריכים למדל ערכים לא סטנדרטיים (X, Z)
- צריכים ארגון מודולים יותר מאשר פונקציות שלהם: סוגי נתונים חדשים, תחביר מותאם אישית, ...

בדיקות של DUT



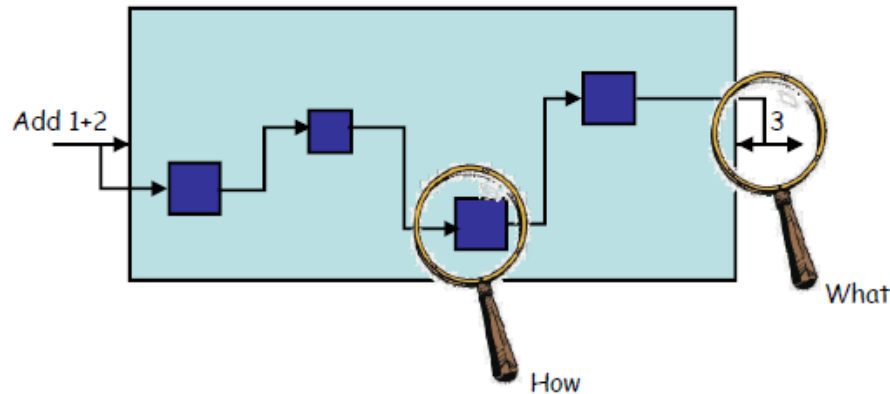
בדיקות אידיאליות

➤ זיהוי של סטייה מן התנהגות הצפויה ברגע שזה קורה
ואיפה שזה קורה

➤ זה לא קל (גם אם נתעלם מהיבטים המעשיים הרבים) כי
במקרים רבים אנו מבינים שמשהו רע קרה רק בדיעבד

➤ ומה לגבי באגים אנחנו לא חיפשנו?

היבטים מעשיים



➤ העלות של יישום ותחזוקה

➤ העלות של ניתוח הבאגים

➤ העלות של טעויות

- לא הצלחנו לזהות באג שנחשף על ידי stimuli
- סימנו בטעות התנהגות טובה כשגויה

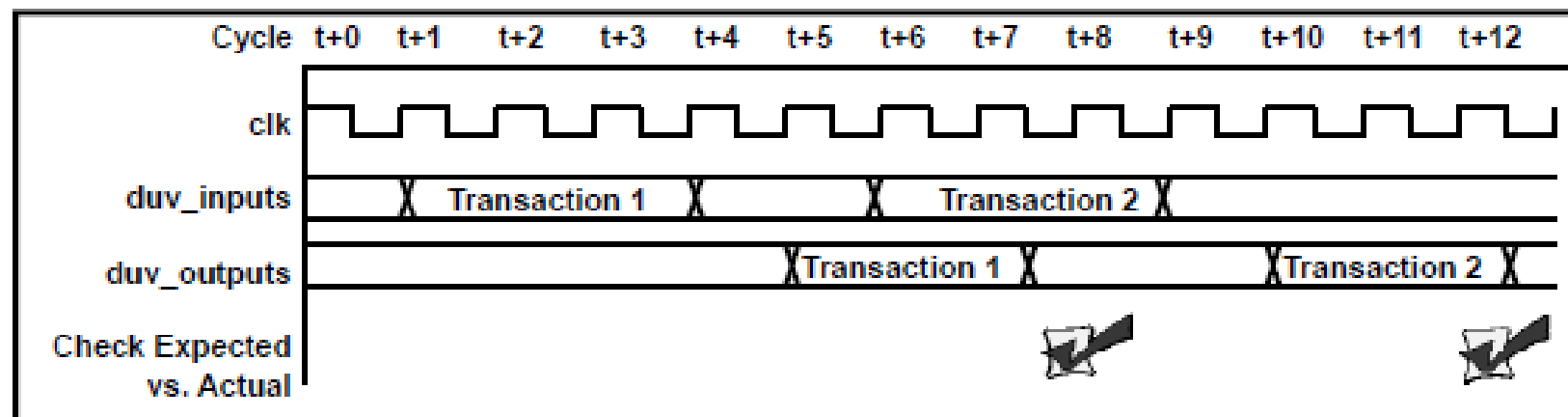
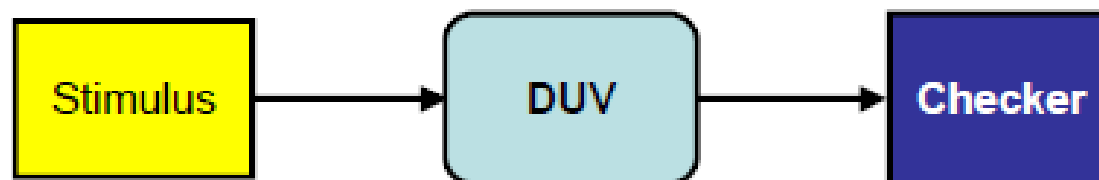
מתי לבדוק?

בדיקות ניתן לעשות בשלבים שונים של תהליך האימות

- במהלך הסימולציה: בדיקות On-the-fly
- בסוף של סימולציה: בדיקות End-of-test
- לאחר תהליך האימות מסתיימים: בדיקות חיצוניים

בדיקות On-the-fly

- Checking is done **while the simulation is running**
- The DUV is continuously monitored to detect erroneous behavior



בדיקות On-the-fly

יתרונות ➤

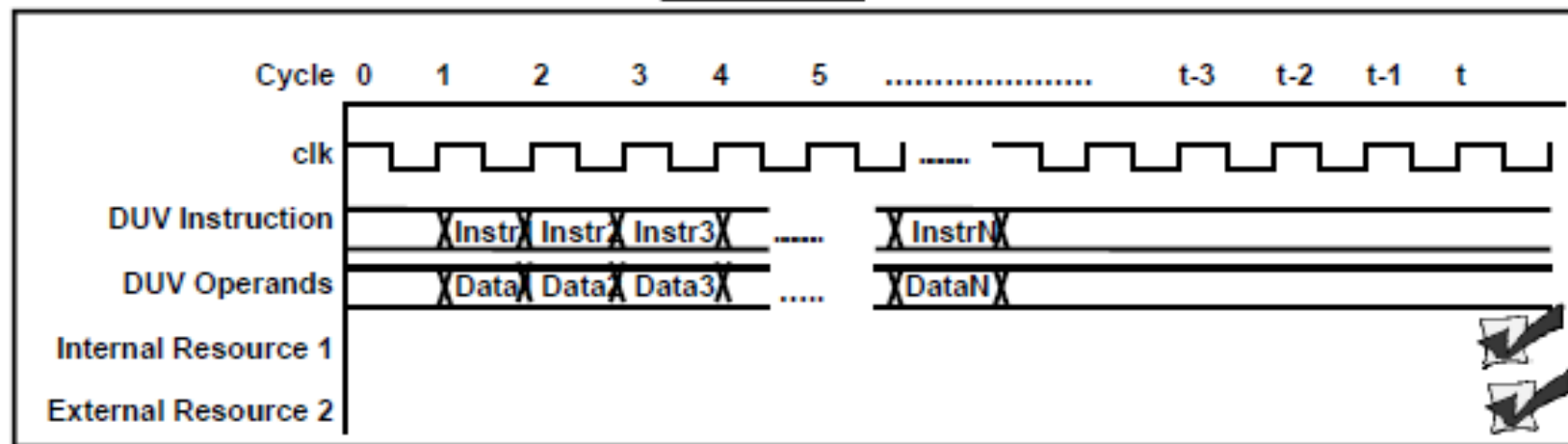
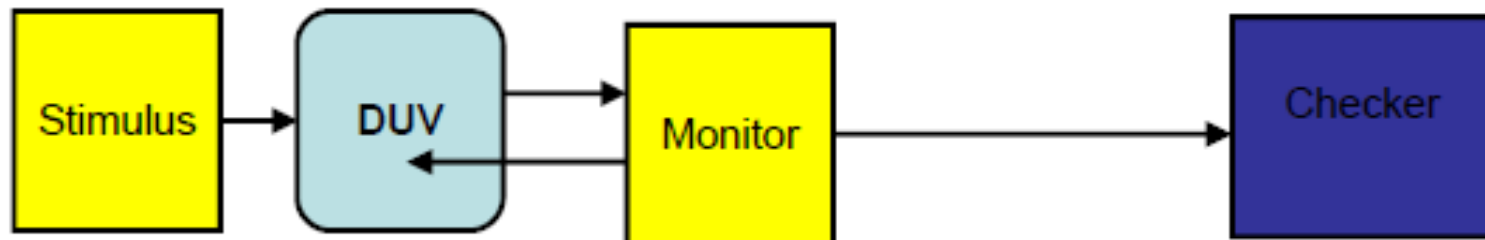
- איתור הבאג יכול להיות קרוב ככל שאפשר (בזמן ובמקום) למקור הבאג
- ניתן לעצור את הבדיקה מוקדם יותר כאשר הבאג מתרחש, לא מבזבזים זמן הסימולציה
- לא דורשים חיפוסים גדולים וכלים חיצוניים לביצוע הבדיקה

חסרונות ➤

- עלול להאיט את הסימולציה
- בדיקת מוגבלות בסיבוכיות הזמן והמקום
- חייבים לתכנן את הבדיקות מראש

בדיקות End-of-test

- Checking is done **at the end of simulation**
- The checker checks the state of internal and external resources and makes sure that they are correct



בדיקות End-of-test

יתרונות ➤

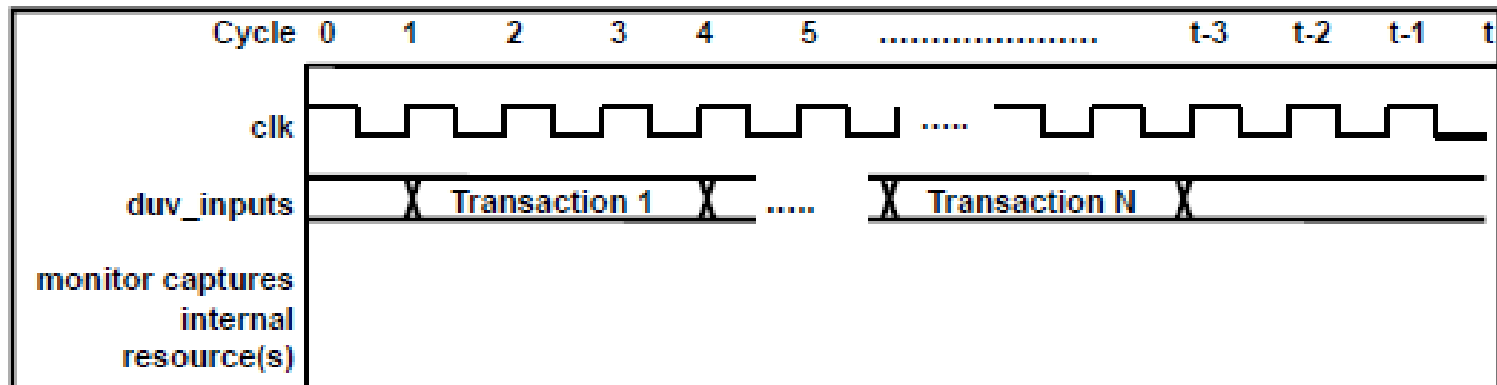
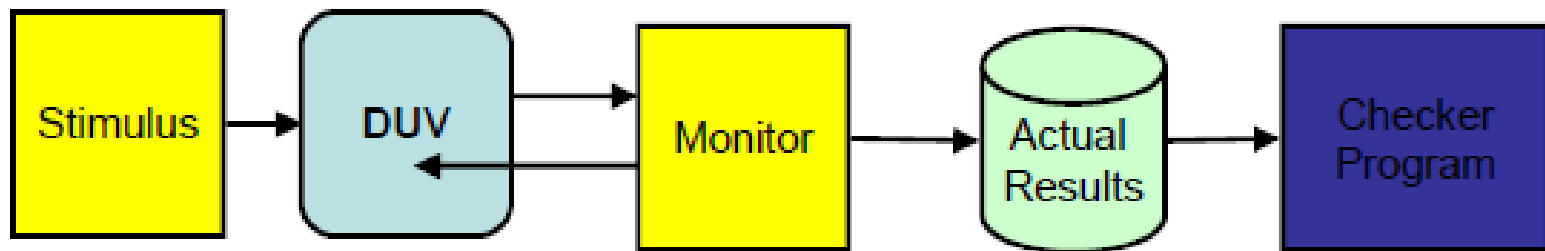
- פשוטות יותר מאשר בדיקות אחרות
- אינן דורשות הבנה עמוקה של DUV
- מפחיתות הסתברות של אזעקות שווא (false alarms)

חסרונות ➤

- מספקות יכולות בדיקה מוגבלות
- קשה לזהות באגים בביצועיים: דברים נכונים קורים, אבל לא בזמן הנכון
- קשה לקשר בין הסימפטומים ומקורותיהם

בדיקות חיצוניות

- **Monitors** keep internal resources values and behaviors in trace files
- Checking is done by an external program that examines these files



בדיקות חיצוניות

➤ בדיקות חיצוניות המפרידות בין בדיקה לבין סימולציה

➤ אנחנו יכולים לבצע כל בדיקה שאנחנו רוצים בלי להריץ מחדש את הסימולציה

➤ אנו יכולים לבצע בדיקות מורכבות יותר: שימוש בהיסטוריה ארוכה יותר, ניתוח אירועים מחוץ לסדר שלהם

➤ אנחנו יכולים לשלב מידע המגיע ממקורות שונים

יצירת גירויים (Stimulus) ובידקות

➤ באופן כללי, בדיקות צריכות להיות מבודדות מן יצירת הגירויים

- מודולריות - יכולת להחליף את גנרטור הגירויים
- שימוש חוזר - היכולת להשתמש בכל הכלים ברמה גבוה יותר של ההירארכית התכנון

➤ גנרטור גירויים יכול לסייע בבדיקות על ידי שיפור של שקיפות

- עוזר להעביר אירועים מפינות חשוכות אל אור הזרקורים

לוח התוצאות -- Scoreboards

➤ לוח התוצאות זה מבני נתונים " חכמים " ששומרים מעקב אחר אירועים שהתרחשו במהלך הסימולציה

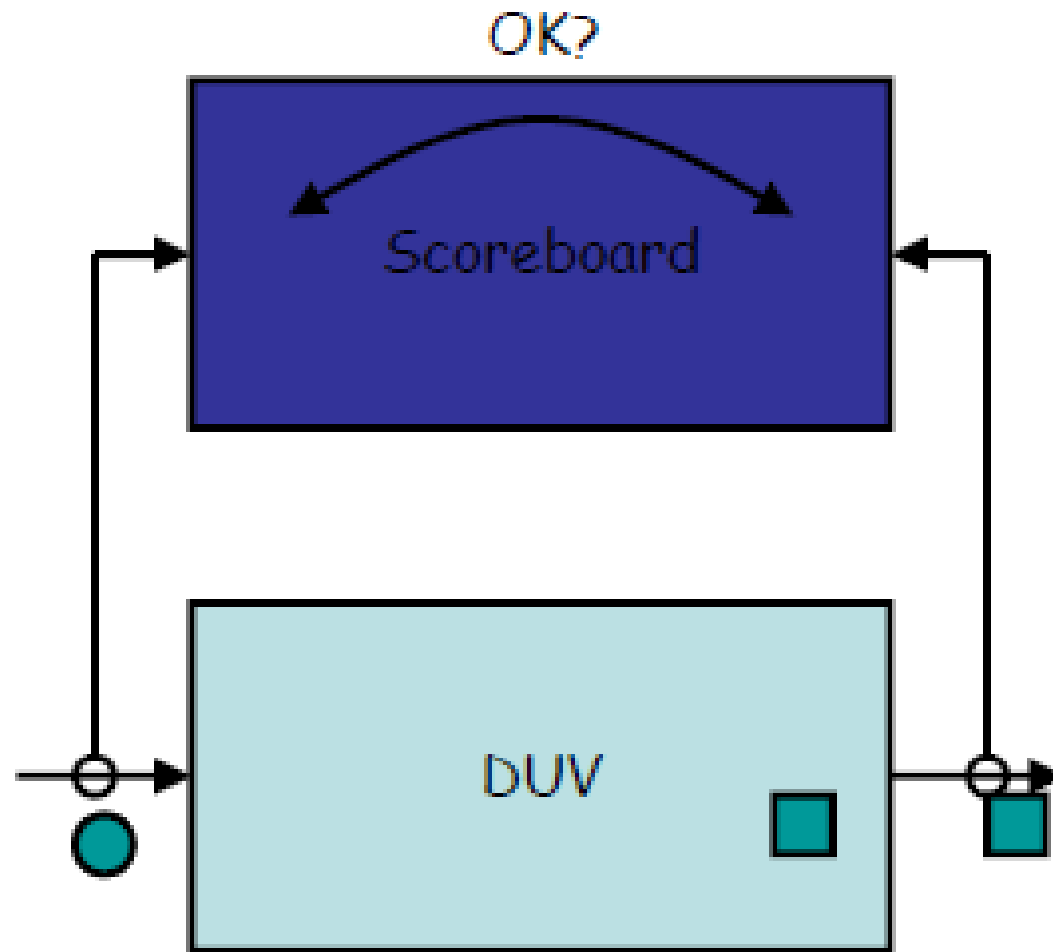
➤ לוח התוצאות לא מבצע בדיקות

➤ המטרה העיקרית של לוח התוצאות היא השימוש שלהם עבור הבדיקות

➤ בפועל, בדיקות רבות מיושמות בתוך לוח התוצאות

➤ ישנן בדיקות טיפוסיות רבות שנעשות בעזרת לוח התוצאות

הפעלת לוח תוצאות



לוח התוצאות

➤ מקורות מידע ללוח התוצאות

- בראש ובראשונה, את קלט ופלט של DUV
- אירועים פנימיים יכולים לשמש גם

➤ סוגי בדיקות נעשים עם לוח התוצאות

- התאמה בין קלט ופלט: שום דבר לא הולך לאיבוד, שום דבר לא נולד, הנתונים תואמים
- חוקי התזמון: עיכוב מקלט לפלט הוא בגבולות המותר
- חוקי הסדר

לוחות התוצאות מאוד שימושיים בתכנוני זרימת הנתונים

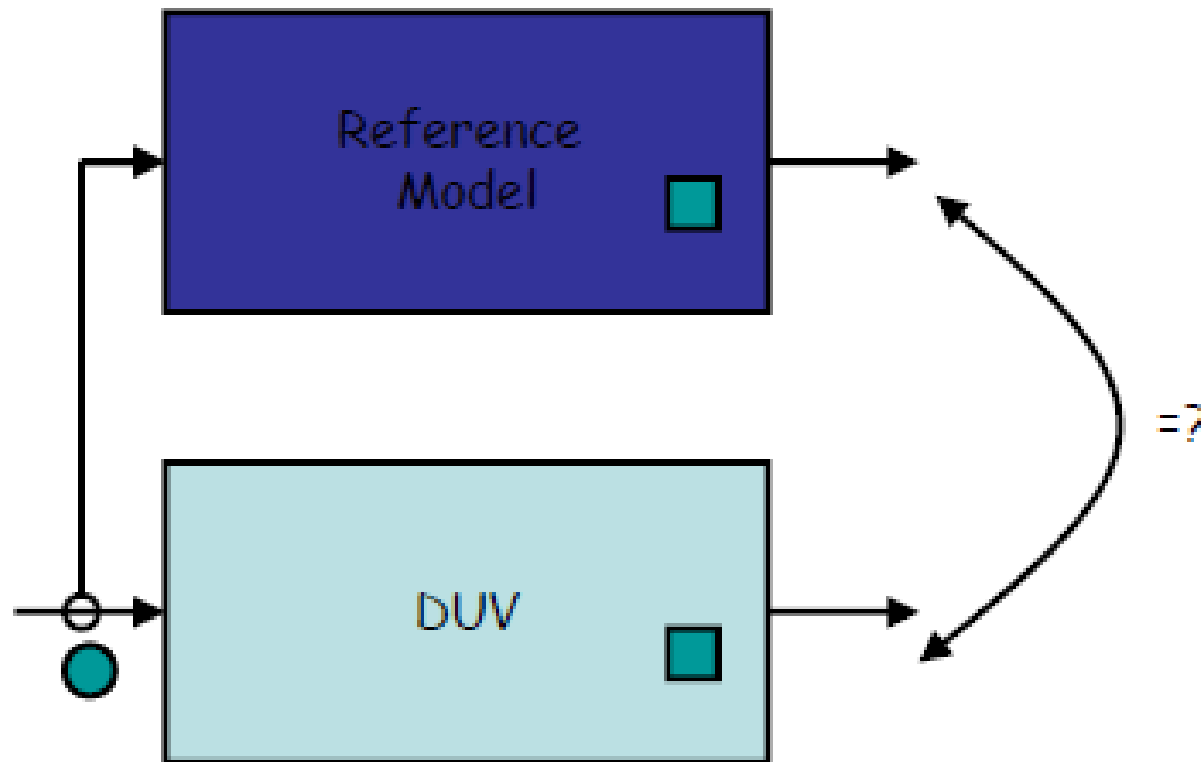
מודלי ההתייחסות -- Reference Models

➤ מודל ההתייחסות הוא אורקל שמספר כיצד DUV צריך להתנהג: בדרך כלל בצורה של יישום חלופי

➤ הוא פועל במקביל ל-DUV תוך שימוש באותו קלט ומספק את מנגנון בדיקות המידע על ההתנהגות הצפויה

➤ מודל התייחסות טהור יכול לרוץ באופן עצמאי מ-DUV

הפעלת מודלי ההתייחסות

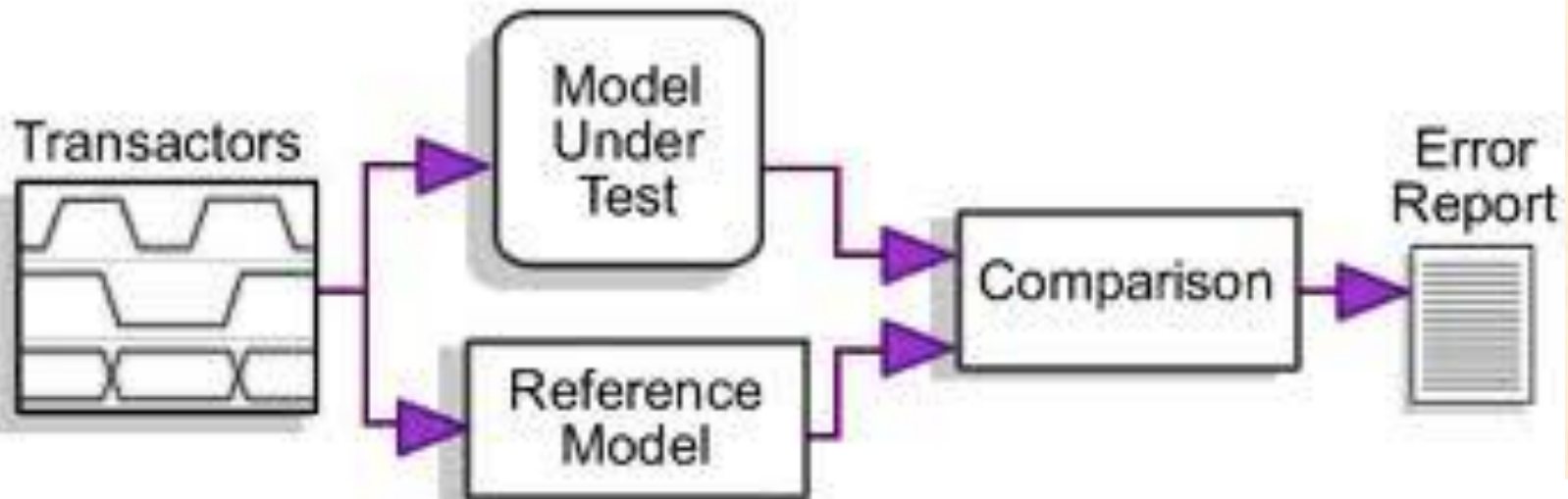


מודלי ההתייחסות

למודלי ההתייחסות יש שימושים רבים

- בדיקות
- עזרה לייצור גירויים
- חיקוי של פונקציה של DUV

Golden Reference Model



בדיקות מבוססות חוקים

➤ החוקים יכולים לבוא ממקורות רבים: מפרט, רמה גבוהה של תכנון, יישום

➤ בדיקות החוקים יכולות להיות מיושמות במקומות רבים: בדיקות בעזרת כלים חיצוניים, במקומות שונים בסביבת האימות, ...

➤ בדיקה מבוססת חוקים המוטבעים בקוד DUV
נקראת הטענות -- assertions